KijaniKiosk Capstone

Track A — Infrastructure-First: Staging Environment, Pipeline Gate, and Alerting

Alan · Capstone Presentation Day

No gate stood between a code change and production

No staging environment

kk-payments changes went straight to production with no isolated place to catch issues first.

No automated health check

Deploys weren't verified before reaching real traffic — a broken build could ship silently.

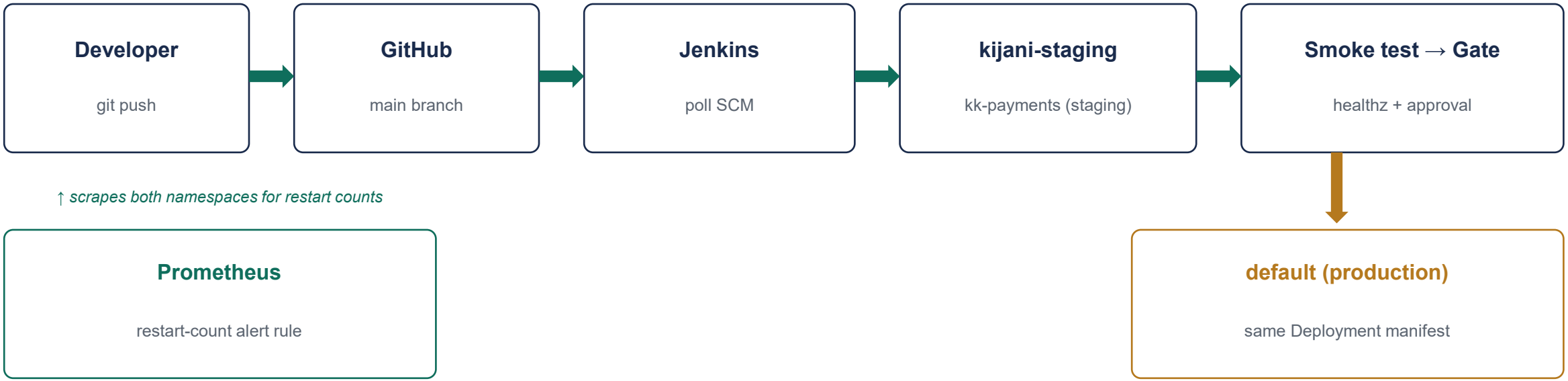
No human checkpoint

There was no required approval step, and no record of who approved a release or why.

This project builds the staging environment, the automated gate, and the alert rule that close that gap.

02 · ARCHITECTURE

One manifest, two environments, one gate between them



Terraform provisions kijani-staging · kustomize applies one shared manifest with per-environment ConfigMaps · Jenkins gates promotion on a passing smoke test

03 · PIPELINE DEMONSTRATION

A full run: staging deploy → smoke test → gate → production

→ 1. Merge to main triggers Jenkins automatically (Poll SCM) — no manual step.

→ 2. kubectl apply -k k8s/staging deploys kk-payments to the isolated namespace.

→ 3. Smoke test curls /healthz over a port-forward; a non-200 fails the build before the gate.

→ 4. Approval gate pauses the pipeline and requires a typed APPROVAL_REASON.

→ 5. On approval, the identical manifest is applied to production and rolled out.

Recorded evidence

- Build #5 recorded a genuine failure
- Build #9 shows the gate paused with a specific, real approval reason referencing the actual smoke test result
- Console output: "deployment kk-payments successfully rolled out" · Finished: SUCCESS

04 · KEY DECISION

Placeholder image vs. blocking on the real application

The trade-off

No production kk-payments image existed in this environment. I could stall infrastructure work until one was rebuilt, or prove the deployment/pipeline/RBAC logic independently of application code using hashicorp/http-echo — a container satisfying the same ports, probes, and ConfigMap wiring a real app would.

Why this was the right call

→ The pipeline mechanics — RBAC, smoke test, approval gate — are independent of application code and fully provable this way

→ Blocking on a rebuilt app would have cost hours with no infrastructure progress to show

→ The gap is named explicitly in the scope document, not hidden

→ Swapping in a real image later is a one-line change to deployment.yaml

Where AI helped, what it got wrong, and how I checked

RBAC binding for Jenkins

What it got wrong: First attempt bound to the wrong ServiceAccount, so kubectl calls failed silently against the real cluster.

Fix & verification: Rebound to the build agent's actual in-cluster ServiceAccount, scoped to kijani-staging + default; verified with a live pipeline run.

Deploy stage guards

What it got wrong: Initial Jenkinsfile had `when { branch "main" }` guards that never matched this repo's actual branch context, blocking every stage.

Fix & verification: Removed the guards; confirmed by triggering a real build that reached the approval gate.

Serverless deploy capability flag

What it got wrong: CloudFormation stack failed validation with a vague error; root cause was a missing CAPABILITY_NAMED_IAM flag, not surfaced by the framework.

Fix & verification: Diagnosed via aws cloudformation validate-template directly; documented as a known Framework v4 quirk.

06 · PRODUCTION GAPS

What's not production-ready, and the specific remediation

Placeholder application image

Gap

kk-payments runs as hashicorp/http-echo, not the real service.

Remediation

Swap the image field in k8s/base/deployment.yaml once a real container is built; no other pipeline change required.

Single-region, no HA

Gap

minikube is a single node; no multi-zone or autoscaling.

Remediation

Move to a managed cluster (GKE/EKS) with a HorizontalPodAutoscaler and pod anti-affinity across zones.

Resource quota is a rough estimate

Gap

The staging ResourceQuota (1 CPU / 1Gi) was not load-tested.

Remediation

Run a load test against staging and tune requests/limits from observed usage before raising to match production sizing.

No production alert coverage

Gap

The Prometheus rule watches restart count only, not latency or error rate.

Remediation

Add the error-rate and latency rules already drafted in monitoring/kk-payments-alerts.yaml once the app exposes real request metrics.

A gate now stands between every change and production.

Terraform · Kubernetes · Jenkins · Prometheus — proven end-to-end, with the gaps named honestly.

github.com/kyos-hub/kijanikiosk-capstone